

Insertion Sort

Insertion Sort (A, n)	Cost	Time
1. for (i = 2 to n)	C_1	n
2. key = A[i]	C_2	n-1
3. j = i-1	C_3	n-1
4. while (j > 0 and j > key)	C_4	$T = t_2 + t_3 + t_4 + t_5 + t_6 \rightarrow n$ 2 2 1 5 4
5. A[j+1] = A[j]	C_5	$T - (n-1)$
6. j = j-1	C_6	$T - (n-1)$
7. A[j+1] = key	C_7	n-1

t_2 - 1 pass 1 fail

t_3 - 1 pass 1 fail

t_4 - 1 fail

t_5 - 4 pass 4 fail

t_6 - 3 pass 1 fail

$$T(n) = C_8T + C_9n + C_{10}$$

DSA-I, Tuesday

30.05.23, class-02

$$T(n) = C_8T + C_9n + C_{10}$$

$$n=5$$

$$i=5, 1 \ 2 \ 3 \ 5 \ 7$$

$$T = 5-1 = n-1$$

$$T = 1+1+1+1$$

$$1 = 2 \ 3 \ 4 \ 5$$

Sorted : Best case

$$T(n) = C_8(n-1) + C_9n + C_{10}$$

$$= an + c$$

$T(n) = an + a$, order of $O(n)$

$i = 2, 5, 4, 3, 1 \quad n = 4 \quad T = 2 + 3 + 4$

$$T(n) = C_8 \left[\frac{n(n+1)}{2} - 1 \right] + C_9 n + C_{10} = 1 + 2 + 3 + \dots + n - 1$$
$$= \frac{n(n+1)}{2} - 1$$

$$= C_8 \left[\frac{n^2}{2} + \frac{n}{2} - 1 \right] + C_9 n + C_{10}$$

$$= \frac{C_8}{2} n^2 + \frac{C_8}{2} n - C_8 + C_9 n + C_{10}$$

$$= \frac{C_8}{2} n^2 + n \left(\frac{C_8}{2} + C_9 \right) + (C_{10} - C_8)$$

$T(n) = an^2 + bn + a$, order of $O(n^2)$

Upper Bound Notation

Run time is in $O(n^2)$

$$f(n) = O(g(n))$$

$$T(n) = O(n^2)$$

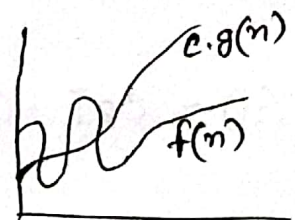
$$an^2 + bn + c = O(n^2)$$

$\downarrow \qquad \qquad \downarrow$
 $f(n) \qquad \qquad g(n)$

$$0 \leq f(n) \leq c \cdot g(n)$$

for some c and n_0

where $n \geq n_0$



Problem:

Show that $f(n) = 10n^2 + 7n + 4 = O(n^2) \rightarrow g(n)$

we have to show $0 \leq f(n) \leq c \cdot g(n)$

$$\rightarrow n \geq n_0, c = 15$$

$$n = 1, f(n) = 10 + 7 + 4 = 21, c \cdot g(n) = 15$$

$$n = 2, f(n) = (10 \cdot 2^2) + (7 \cdot 2) + 4 = 58, c \cdot g(n) = 60$$

$$n = 3, f(n) = (10 \cdot 3^2) + (7 \cdot 3) + 4 = 115, c \cdot g(n) = 135$$

Problem:

$$\text{Show that } f(n) = 10n^2 + 4n = o(n^3)$$

$$= 0 \cdot n^3 + 10n^2 + 4n$$

$$c = 5$$

$$n = 1, f(n) = 10 \cdot 1^2 + 4 \cdot 1 = 14, c \cdot g(n) = 5$$

$$n = 2, f(n) = 10 \cdot 2^2 + 4 \cdot 2 = 48, c \cdot g(n) = 40$$

$$n = 3, f(n) = 10 \cdot 3^2 + 4 \cdot 3 = 102, c \cdot g(n) = 135$$

$$* o(n^2) + o(n^3) + o(n)$$

$$= o(n^3) + o(n^3) + o(n^3)$$

$$= o(n^3)$$

Lower Bound Notation

$$\text{problem: Show that } 5n^2 + 6n + 4 = \Omega(n^3) \checkmark$$

$$= \Omega(n) \times$$

$$5n^2 + 6n + 4 = \Omega(n) \checkmark$$

$$= \Omega(n^3) \times$$

$$= \Omega(1) \checkmark$$

$$= \Omega(n^2) \checkmark$$

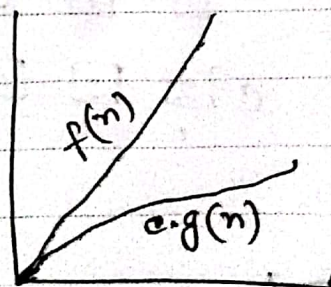
$$0 \leq c \cdot g(n) \leq f(n)$$

$$5n^2 + 6n + 4$$

$$\text{here, } c = 2,$$

$$\therefore n = 1, 5 + 6 + 4 = 15, c \cdot g(n) = 2$$

$$n = 2, 5 \cdot (2)^2 + 6 \cdot 2 + 4 = 36, c \cdot g(n) = 8$$



Asymptotic Tight Bound Notation:

$$f(n) = 5n^2 + 6n + 3$$

$$= \Theta(n^2)$$

$$f(n) = 5n^2 + 6n + 3$$

$$= \Omega(n^2)$$

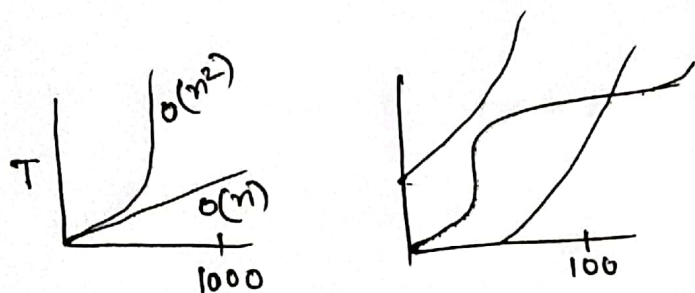
Problem: Show that, $f(n) = 5n^2 + 3n + 2 = \Theta(n^2)$

$$5, 3 \leftarrow = O(n^2)$$

$$2, 1 \leftarrow = \Omega(n^2)$$

another way - c_1, c_2, n

$\downarrow$ $\downarrow$ $\downarrow$
 5 2 3



DSA-I, Saturday
03.06.23, class-03

Counting Sort:

No negative numbers, fixed size

A

2

5

3

0

2

3

0

3

B

--

--

--

--

--

--

--

--

0

0

0

0

0

0

1

1

1

1

1

1

C

2

2

4

7

7

8

Counting Sort Algorithm:

CountingSort(A, B, k, n)

- ① let $C[0 \dots k]$ be a new array
- ② for $i = 0$ to n
- ③ $C[i] = 0$
- ④ for $j = 1$ to 8
- ⑤ $C[A[i]] = C[A[i]] + 1$
- ⑥ for $i = 1$ to k
- ⑦ $C[i] = C[i-1] + C[i]$
- ⑧ for $i = n$ down to 1
- ⑨ $B[C[A[i]]] = A[i]$
- ⑩ $C[A[i]] = C[A[i]] - 1$



Time complexity

$$T(n) = an + bk + c$$

$$T(n) = O(n+k)$$

if $k = O(n)$

$$T(n) = O(n)$$

$$T(n) = an + bk + c$$

$$O(n+k)$$

$$k = O(n) \quad k = O(n)$$

$$k = c_1n + c_2$$

$$O(n + c_1n + c_2)$$

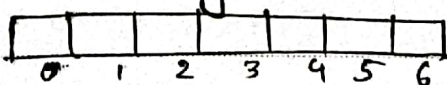
$$O(n)$$

Array Mapping

Mapping 1D Arrays

Mapping 2D Arrays

int A[7]



$$A[\underset{l_1}{0} \dots \underset{u_1}{6}] \quad \text{addr}(A[\overset{i}{3}]) = \underset{\substack{\downarrow \\ \text{base address}}}{b} + (i - l_1) \times L \rightarrow k$$

base address

$$= 10 + (3 - 0) \times 4 = 22$$

00 ... 00
00 ... 00
0000 0000
0001 0001

base address = 10

double - 8 bytes $A[100]$;

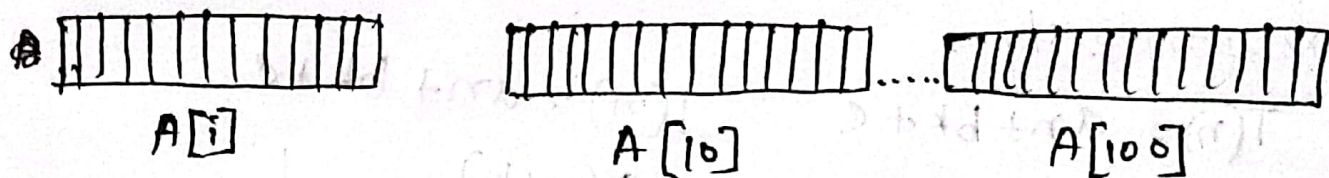
$$A[\underset{l_1}{1} \dots \underset{u_1}{100}]$$

Given,

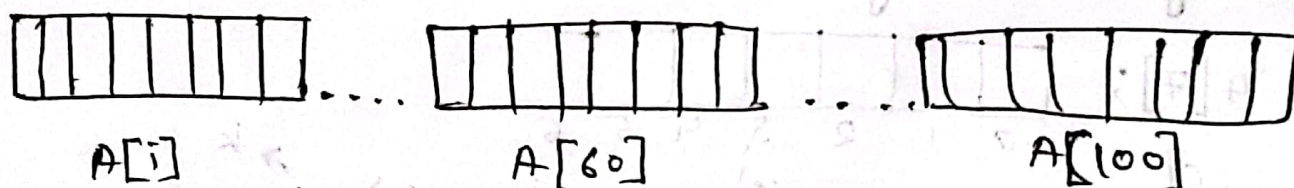
$$\text{addr}(A[10]) = 5CDE16$$

Find $\text{addr}(A[0])$

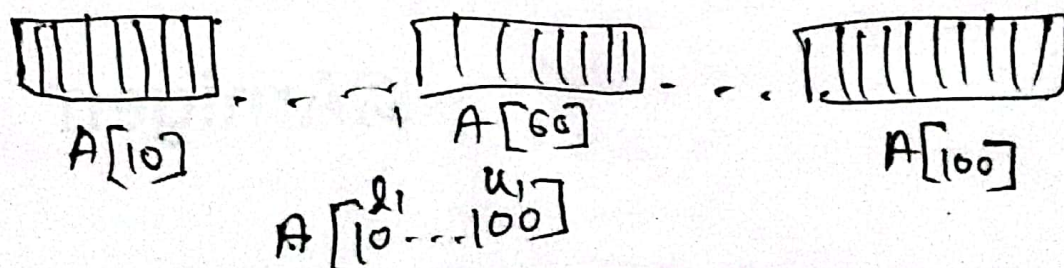
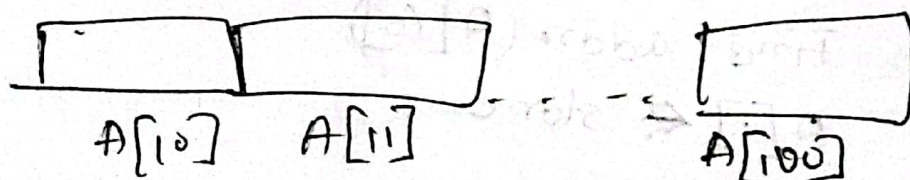
$A[i] \leftarrow \text{store}$



$$\begin{aligned}
 \text{addr}(A[10]) &= b + (i - l_1)L \\
 &\Rightarrow b \\
 &= (i - l_1)L + \text{addr}(A[10]) - (i - l_1)L \\
 &= 5CDF_{16} - (10 - 1)8 \\
 &= 23775 - 72 \\
 &= 23703 \\
 &= 5C97_{16}
 \end{aligned}$$



$$\begin{aligned}
 \text{Addr}(A[60]) &= 6 + (i - l_1) \times L \\
 &= 5C97_{16} + (60 - 1) \times 8 \\
 &= 5C97_{16} + 50 \times 8 \\
 &= 24175 \\
 &= 5E6F_{16}
 \end{aligned}$$



$$\begin{aligned}
 \text{addr}(A[60]) &= b + (i - l_1) \times L \\
 &= 5\text{CDF}16 + (60 - 10) \times 8 \\
 &= 23775 + 50 \times 8 \\
 &= 24175 \\
 &= 5\text{E6F}_{16}
 \end{aligned}$$

Mapping 2D Arrays.

Row Major Order:

$$\text{addr}(a[i, j]) = b + (i - l_1) \times (u_2 - l_2 + 1) \times L + (j - l_2) \times L$$

$$A \begin{matrix} [0 \dots 2] \\ l_1 \quad u_1 \end{matrix} \begin{matrix} [0 \dots 3] \\ l_2 \quad u_2 \end{matrix}$$

$$\begin{aligned}
 \text{addr}(a[1][2]) &= b + (i - l_1) \times (u_2 - l_2 + 1) \times L + (j - l_2) \times L \\
 &= 10 + (1 - 0) \times (3 - 0 + 1) \times 4 + (2 - 0) \times 4 \\
 &= 10 + 1 \times 4 \times 4 + 8 \\
 &= 34
 \end{aligned}$$

2D Array Mapping

13 June 2023-ET-1

int A[3][4]

int A[$\underset{l_1}{0} \dots \underset{u_1}{2}$][$\underset{l_2}{0} \dots \underset{u_2}{3}$]

$$a[i][j] = b + (i - l_1)(u_2 - l_2 + 1) \times L + (j - l_2) \times L$$

$$= 10 + (1 - 0)(3 - 0 + 1) \times 4 + (2 - 0) \times 4$$

$$= 34$$

$$a[1, 2] = c\#$$

$$a[1][2] = c++$$

Col Major Order:

A[1... 3][1... 4]

$$\text{addr}(A[i][j]) = b + (j - l_2) \times (u_1 - l_1 + 1) \times L + (i - l_1) \times L$$

$$= 10 + (2 - 1) \times (3 - 1 + 1) \times 4 + (1 - 1) \times 4$$

$$= 22$$

array of i

a[l₁... u₁]

$$a[i] = b + (i - l_1) \times L$$

$$a[i][j] = b + (i - l_1)(u_2 - l_2 + 1)L + (j - l_2)L \rightarrow \text{Row}$$

3D Array

$$a[l_1 \dots u_1][l_2 \dots u_2][l_3 \dots u_3]$$

$$a[i][j][k] = b + \underbrace{(i-l_1)(u_2-l_2+1)(u_3-l_3+1)L + (j-l_2) \times (u_3-l_3+1)L + (k-l_3)L}_{\text{Row}}$$

4D Array

$$a[l_1 \dots u_1][l_2 \dots u_2][l_3 \dots u_3][l_4 \dots u_4]$$

$$a[i][j][k][x] = b + \underbrace{(i-l_1) \times (u_2-l_2+1) \times (u_3-l_3+1) \times (u_4-l_4+1)L + (j-l_2) \times (u_3-l_3+1) \times (u_4-l_4+1)L + (k-l_3) \times (u_4-l_4+1)L + (x-l_4)L}_{\rightarrow \text{Row}}$$

Linear Search

```
int LinearSearch(int b[], int skey) {
```

```
    int i;
```

```
    for (i=0; i < size; i++)
```

```
        if (b[i] == skey)
```

```
            return i;
```

```
    return -1;
```

```
}
```

Time Complexity

$$\begin{aligned}T(n) &= C_1(n+1) + C_2n + C_31 \\&= O(n) \\&= \Omega(n) \\&= \Theta(n)\end{aligned}$$

counting sort always
linear time in graph

$$\begin{aligned}\text{Best case: } T_b(n) &= C_1 + C_2 + C_3 = O(1) \\&= O(1) \\&= \Omega(1) \\&= \Theta(1)\end{aligned}$$

Average Case:

Arg. time = $\frac{\text{Time requires for all possible cases}}{\text{Total numbers of cases}}$

$$\begin{aligned}&= \frac{(1+2+3+\dots+n)C}{n} \\&= \frac{\frac{n(n+1)}{2} \times C}{n} = \frac{(n+1)}{2} \times C = O(n)\end{aligned}$$

Binary Search:

at first binary search should be sorted

pseudocode iteratively

int BinarySearch(int A[], int key, int n, int key)

1. $i = 1, j = n$

2. while ($i \leq j$)

$$\text{mid} = \frac{i+j}{2}$$

3. $mid = \left\lfloor \frac{i+j}{2} \right\rfloor$

4. if ($A[mid] == key$)

5. return mid

6. else if ($A[mid] < key$)

7. $i = mid + 1$

8. else if ($A[mid] > key$)

9. $j = mid - 1$

10. return -1

pseudocode recursively

1. RBinSearch (A, l, h, key)

2. if ($l > h$)

3. return -1

4. else

5. $mid = \left\lfloor \frac{l+h}{2} \right\rfloor$

6. if $key == A[mid]$

7. return mid

8. if $A[mid] < key$

9. return RBinSearch ($A, mid+1, h, key$)

10. if $A[mid] > key$

11. return RBinSearch ($A, l, mid-1, key$)

Time Complexity

Best case : $T(n) = c = O(1)$

Worst case: Let us consider the no. of times the loop will run.

Avg si

Array size considered $\frac{n}{2^0} - 1^{st}$

Array size considered $\frac{n}{2^1} - 2^{nd}$

Array size: $\frac{n}{4}$ or $\frac{n}{2^2} - 3^{rd}$

Array size: $\frac{n}{8}$ or $\frac{n}{2^3} - 4^{th}$

Array size: $\frac{n}{2^k} = 1 - (k+1)$

$$\frac{n}{2^k} = 1$$

$n = 2^k$

$$\log_2 n = \log_2 2^k = k \log_2 2$$

$$= k = \log_2 n$$

while loop run for $(n+1)$ times. or.

$$\log n + 1, O(\log n)$$

Time Complexity Analysis:

$$\text{Avg. time req.} = \frac{\text{time needed for all possible cases}}{\text{total no. of cases}}$$

while loop will run 1 time for 1 element $\rightarrow \frac{n}{2} \text{ pos}^n - 1 \text{ element}$

while loop will run 2 times for 2 elements $\rightarrow \frac{n}{4} \text{ pos}^n, \frac{3n}{4} \text{ pos}^n - 2 \text{ element}$

while " " " 3 times for 4 elements $\rightarrow \frac{n}{8}, \frac{3n}{8}, \frac{5n}{8}, \frac{7n}{8} \rightarrow 4 \text{ element}$

while loop " " $\log_2 n$ " " $2^{\log_2 n - 1}$ elements

Total time for all possible cases:

$$1 \times 1 + 2 \times 2 + 4 \times 3 + \dots + 2^{\log_2 n - 1} \times \log_2 n$$

$$\text{Avg. time req.} = \frac{n \times (\log_2 n + 1) + 1}{n}$$

$$\approx \frac{n \times (\log_2 n - 1) + 1}{n}$$

$$= \log_2 n - 1$$

$$= O(\log_2 n)$$

Merge Sort

Merging Two Decks of Cards

[2] [8] [15] [18]

[5] [9] [12] [17] [20] [23]

[2] [5] [8] [9] [12] [15] [17] [18] [20] [23]

Merging in an Array

[] [2] [4] [5] [7] [] [1] [2] [3] [6] []

L: [2] [4] [5] [7] [] R: [1] [2] [3] [6] []

Merge Sort Pseudocode

MERGE-SORT(A, p, n)

- ① if $p < n$
- ② then $q \leftarrow \lfloor (p+n)/2 \rfloor$
- ③ MERGE-SORT(A, p, q)
- ④ MERGE-SORT($A, q+1, n$)
- ⑤ MERGE-SORT(A, p, q, n)

Running Time

$$T(n) = b, n = 1$$

$$= T(n/2) + T(n/2) + bn$$

$$= 2T(n/2) + bn$$

$$= 2 \left[2T(n/4) + \frac{bn}{2} \right] + bn$$

$$= 4T(n/4) + 2bn$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2bn$$

$$= 2^2 \left[2T\left(\frac{n}{8}\right) + \frac{bn}{4} \right] + 2bn$$

$$= 2^3 T(n/8) + 3bn$$

$$= 2^3 T(n/2^3) + 3bn$$

$$= 2^4 T(n/2^4) + 4bn$$

$$= 2^i T(n/2^i) + i bn$$

$$= \boxed{2^i b + i bn}$$

$$\frac{n}{2^i} = 1$$

$$\text{or, } n = 2^i$$

$$\text{or, } \log_2 n = \log_2 2^i$$

$$\text{or, } i = \log_2 n$$

$$\Rightarrow 2^i b + i bn$$

$$= nb + \log_2 n bn$$

$$= bn \log_2 n + nb$$

$$= O(n \log_2 n)$$

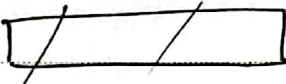
$$= \Theta(n \log_2 n)$$

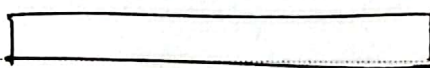
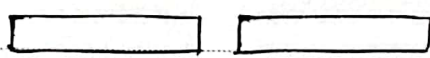
Merge Pseudocode

Merge (A, p, q, r)

- ① $n_1 = q - p + 1$ $O(1)$
- ② $n_2 = r - q$ $O(1)$
- ③ Let $L[n_1+1]$ and $R[n_2+1]$ $O(n)$
be two new arrays.
- ④ For $i = 1$ to n_1 $O(\frac{n}{2})$
- ⑤ $L[i] = A[p-1+i]$
- ⑥ for $i = 1$ to n_2 $O(\frac{n}{2})$
- ⑦ $R[i] = A[q+i]$
- ⑧ $L[n_1+1] = \infty$ $O(1)$
- ⑨ $R[n_2+1] = \infty$ $O(1)$
- ⑩ $i = 1$ $O(1)$
- ⑪ $j = 1$ $O(1)$
- ⑫ For $k = p$ to r $O(n)$
- ⑬ if $L[i] \leq R[j]$ $O(1)$
- ⑭ $A[k] = L[i]$ $O(1)$
- ⑮ $i = i + 1$ $O(1)$
- ⑯ if $R[j] < L[i]$
- ⑰ $A[k] = R[j]$
- ⑱ $j = j + 1$

Merge Sort: Running Time (Recursion Tree)

~~depth~~ ~~1's~~ size 0  ~~bn~~
~~1~~ ~~1~~ ~~1st~~ 2⁰

0  bn
1st 2⁰ n/2  $\frac{bn}{2} + \frac{bn}{2} = bn$

2¹ n/2¹  $\frac{bn}{4} + \frac{bn}{4} + \frac{bn}{4} + \frac{bn}{4} = bn$

¹
2¹ n/2¹ 

$$\begin{aligned}
& 2^i bn + bn \\
& = \log_2 n \, bn + bn
\end{aligned}$$